CHANGA TUTORIAL: RUNNING A PATCH OF THE UNIVERSE WITH THE CHANGA CODE.

Introduction

In this tutorial you will generate the initial condition for a cosmological simulation, perform the simulation yourself using the public version of the ChaNGa code, and analyse it. The simulation has quite low resolution, so you can run it on your laptop. We have a docker to produce the simulation environment required for this tutorial, which can be downloaded at (https://drive.google.com/drive/folders/1zybgBOSOL3s7Yvwdm-kAczjhO6–ru4R). At the start of the tutorial we will go through how to run the docker. Thereafter we will show how we compile ChaNGa and Charm++. Then we will produce an initial condition using the MUSIC software. We will then go through how the parameter file works in ChaNGa and what run instrucitons are available.

We will also go through two popular analysis tools to visualize the data. One is splash a powerful and light-weight interactive visualization software for particle codes (works for both ChaNGa and OpenGadget). We will also use the Pynbody, which is specialized to visualize and perform analysis of cosmological simulations. Pynbody can make use of halo-finder algorithms like AHF (Amiga Halo Finder) to identify the dark matter halos. Which can be used to produce global properties of the halo population and local profiles of the specific halos.

1. Running the docker

First download the docker container at , if you have not already done so. Be sure to download the docker container for your cpu architecture. First we need to load the docker from the downloaded .tar file

FOR ARM ARCH

```
docker load -i changa-container-arm.tar
```

FOR X86_64 ARCH

```
docker load -i changa-container-x86_64.tar
```

After sucessfully loading the image into your docker. We can run the docker container by using the following command:

FOR ARM ARCH

```
docker run --rm -it -v $(pwd):/output \
changa:arm64 /bin/bash
```

```
FOR X86_64 ARCH
docker run --rm -it -v $(pwd):/output \
    changa:arm64 /bin/bash
```

For interactive visualization when using docker you require XQuartz, with the following extra commands to the docker run script above:

```
FOR MAC
-e DISPLAY=host.docker.internal:0 \

FOR LINUX (not fully tested)
-e DISPLAY=$DISPLAY -v /tmp/.X11-unix:/tmp/.X11-unix \
```

Within the container, all of the required software has already been compiled but for educational purposes we will go through how to build and compile ChaNGa in the following sections.

2. Compilation of Charm++ and ChaNGa

In your home folder within the docker environment, you will find the folders ChaNGa, charm and utility. These are all software downloaded from git, using the following git clones:

```
git clone https://github.com/N-BodyShop/changa.git ChaNGa
git clone https://github.com/N-BodyShop/utility.git
git clone https://github.com/UIUC-PPL/charm.git
```

When building Charm++, three important choices are to be made, the first is what communication layer should be used, second is what CPU architecture you are using(x86_64,arm etc.), and third to use CUDA or not. If building for HPC machines it is also important to run with SMP mode. SMP refers to Symmetric Multi-Processing, which means many cores on each compute node have access to the same memory space. The charm run time can take advantage of this access and use fewer messages, but the start command needs to be modified to tell ChaNGa about the processor configuration. The communication layer abstracts the underlying communication infrastructure, providing the flexibility to use either high-performance, machine-specific primitives or general-purpose standards such as MPI. Several options are available, but for multi-node architectures the default choice is typically either MPI/UCX or verbs. The MPI option relies on an underlying MPI library, which performs communication through the UCX network layer. Alternatively, one can build directly with the UCX option to avoid the intermediate MPI layer; however, due to common HPC system configurations, this approach can be more difficult to set up reliably, and MPI is therefore generally preferred. The verbs communication layer communicates directly with the InfiniBand network using low-level libraries such as OpenFabrics or ibverbs, which in many cases offer the best performance. For this tutorial, we use the multicore

option, which is designed for shared-memory communication on a single node (check last section for HPC setup if intrested).

```
FOR ARM ARCH
        ./build ChaNGa multicore-linux-arm8 --with-production -j8 --force;
FOR x86_64 ARCH
        ./build ChaNGa multicore-linux-x86_64 --with-production -j8 --force;
```

Now after Charm++ has been built we can move on to configuring and compiling ChaNGa. ChaNGa can be configured and compiled by the following command:

```
./configure --enable-avx --enable-cooling=metal --enable-splitgas --enable-wendlanc
make -j8
```

Here `--enable-avx` enables avx instruction, `--enable-cooling=metal` enables the metal radiative cooling module (there are different libraries to choose from), `--enable-splitgas` allows gas particles to split when they get too heavy and `--enable-wendland` uses the Wendland C4 kernel.

3. Generating a cosmological initial condition

We will use a popular initial condition generator called MUSIC (Multi-scale initial conditions) (Hahn and Abel (2011)). MUSIC creates particle (or grid) positions and velocities that represent the early Universe at high redshift, consistent with a chosen cosmology and power spectrum (density fluctuations). MUSIC can also generate nested grid initial conditions for high-resolution "zoom-in" cosmological simulations, to increase the resolution in specific regions. In this tutorial we use a single grid as our initial conditions and not use the zoom-in feature. MUSIC can output initial conditions for many codes, which include the SPACE codes RAMSES, ChaNGa and OpenGadget. The software is already downloaded and compiled in the docker environment. We have already prepared a configuration file, which will generate an IC with $L = 20$ Mpc with $2 * 32^3$ particles at $z = 127$, with both a dark matter and baryon component. We will use Planck cosmology parameters where $\Omega_m = 0.3077$, $\Omega_L = 0.6923$ and $H_0 = 67.81$ km/s/Mpc. `L20_32_eisenstein_ic.conf`. To run MUSIC with the prepared configuration file you can simply run:

```
Inside the TutorialFiles folder:
../music/MUSIC L20_32_eisenstein_ic.conf
```

This will generate the initial condition file L20_32_IC.dat.

4. Performing N-body simulation

In this step each of you will evolve the initial condition you generated to the current epoch (i.e. $z = 0$) using the ChaNGa code. First we can have a look into the prepared parameter file L20_32_dark.param within the TutorialFiles folder. I will go through some of the

parameters here, details about other parameters can be found here: https://github.com/N-BodyShop/changa-gasoline-docs/wiki/Runtime-Parameters [1].

```
achInFile = name of initial condition file
achOutName = name of output fil
iCheckInterval  = interval of checkpoint files (1= every global step)
iOutInterval    = interval of output files
iLogInterval    = interval of log output
dRedTo          = run to this redshift (for cosmological sims)
nSteps          = number of global steps
dMsolUnit       = Code unit to Msol unit (rho_crit=1 G=1 for code units)
dKpcUnit        = Code unit to Kpc unit (box length in code units = 1 for cos
bComove = (1=cosmological/expanding 0=non cosmological)
dFracNoDomainDecomp = if fraction of active particles are less than this do n
bDoGravity = enable graviational forces
bDoGas = enable hydrodynamics
bGasCooling = enable radiative cooling
bUV = enable
nSmooth = number of neighbours for SPH
bStarForm = enable star formation
dCStar = starformation efficency
dDeltaStarForm = timestep criterion for star formation
dInitGasMass = initial gas mass, used for particle splitting around this gas
bFeedBack = enable SN feedback
```

As you can see from the prepared parameter file, we have disabled all extra physics except for gravity. We will begin by running and visualizing a simulation run with just gravity. Now we are ready to run ChaNGa, we will use the following command to run ChaNGa:

```
Inside the TutorialFiles folder:
../ChaNGa/charmrun +auto-provision ../ChaNGa/ChaNGa -b 24
+useckloop -D 1 +balancer MultistepLB_SFC -p 512 L20_32_dark.param
```

I will go through each of the command arguments in this run script. charmrun is the charm++ launcher, responsible to launching processes. For the mpi communication layer the charmrun is simply a wrapper that calls mpirun. The argument "+auto-provision" is automatic resource provision, where charm++ decides how many worker processes (PEs) to create based on available hardware. Then we have the ChaNGa executable (../ChaNGa/ChaNGa). The arguments following the ChaNGa exectuable is related to performance. The argument "-b 24" sets the number of partilces in each bucket. The argument "-D 1" sets the domain decomposition to oct-tree. The argument "+balancer

[1] A helpful resource on code unit can be found here https://github.com/N-BodyShop/changa-gasoline-docs/wiki/Code-Units.

MultistepLB_SFC" sets the load balancer to be used (stratergy for assigning pieces to processors). These can be unset and default values usually perform well. Then we have the argument "-p 512", which is probably the most important parameter to configure for ChaNGa as this determines the over-decomposition of your simulation (how many tree pieces/work objects/chares). As default ChaNGa will set the number of tree pieces to be 8 times the number of PEs. However, it is usually better for load balancing (especially when going to higher node counts) to set the number of tree pieces to number of partilces / 1000, such that on average each tree piece has 1000 particles. The final argument must be the parameter file for your simulation. After running the simulation you will see 3 files per each snapshot, the main snapshot looks like `snapshot.000xxx`, which contains majority of the particle data. The `snapshot.000xxx.den` gives you the density, while the `snapshot.000xxx.iord` file contains a unique ID for each particle which is useful for halo finding.

5. Visualization and Data Analysis

To perform the data analysis and visualization, I will showcase two different tools. These are the SPLASH tool, for quick and interactive visualization of particle codes (works also for OpenGadget I think) and the second tool is Pynbody(https://github.com/pynbody/pynbody), which is specialized in analyzing cosmological simulations. Both of these tools are already installed and configured in the docker environment. We will start with using the SPLASH tool. We recommend setting up XQuartz (see section 1) to use splash interactively, but we will also produce some files that can be put in the output folder for viewing. The SPLASH tool can be used with the command "splash filename". Here we will use the following command to load in all the simulation outputs.

```
Inside the TutorialFiles folder:
splash L20_32_dark*[0-9][0-9]
```

This will show the following:

```
 read splash.defaults
         128  filenames read from command line
>>>>>>>>>>>>>>>>>>>>>>>>>>>> L20_32_dark.000001 <<<<<<<<<<<<<<<<<<<<<<<<<<<<
 reading binary tipsy format: big endian:  time:        0.04 ndim: 3
 ntot:       65536 ngas:       32768 ndark:       32768 nstar:
0
 WARNING: fixed softening lengths detected: simulation may contain artificial fragm
        : creating SPH smoothing lengths using h =  1.20*(m/rho)**(1/3)
 >> end of dump file: nsteps =            1 ntot =        65536

  warning: density                 min = max =  0.00E+00
 read splash.limits
```

```
You may choose from a delectable sample of plots
-------------------------------------------------------------------------
   1) x                          6) v_y
   2) y                          7) v_z
   3) z                          8) log density
   4) particle mass              9) log temperature
   5) v_x                       10) h
-------------------------------------------------------------------------
  M) multiplot [   4 ]       m) set multiplot
-------------------------------------------------------------------------
d(ata) p(age) o(pts) l(imits) le(g)end h(elp)
r(ender) v(ector) x(sec/rotate) s(ave) q(uit)
-------------------------------------------------------------------------
Please enter your selection now (y axis or option):2
(x axis) (default=1): 1
(render) (0=none) ([0:10], default=0): 8
(vector plot) (0=none, 5=v) ([0:5], default=0):
Graphics device/type (? to see list, default /xw):density.mp4
```

using the arguments as shown above will generate a movie named density.mp4. You can also just use the default to visualize it in XQuartz. SPLASH has a lot of options and features which can be explored in the documentation (https://splash-viz.readthedocs.io/en/latest/intro.html). You can move the produced .mp4 file to the /output folder to put it outside of the docker system.

For additional data analysis of cosmological simulations it is useful to use a halo finder on your simulation, which identify gravitationally bound structures (dark matter haloes and subhaloes). Here we will use the AMIGA halo finder to identify dark matter halos. The Amiga halo finder is already compiled and an input file performing halo finding on the redshift 0 file has been prepared. Run it with the following script:

```
Inside the TutorialFiles folder:
../ahf/bin/AHF-v1.0-110 AHF.000128.input
```

The halo finder outputs a so called halo catalogue, which gives you all the information on halo mass, virial radius, halo center, etc.. This halo catalogue can be directly loaded into PYNBODY. Now we will use PYNBODY to analyze the redshift 0 data, I have prepared a python file that can be run to plot some results, such as the dark matter halo mass function (HMF) and the dark matter density profile of the most massive halo in your simulation. The python file can be run with the following command:

```
Inside the TutorialFiles folder:
python space_dark.py
```

This will show (make) three figures, a column density render, density profile of most massive halo vs the predicted NFW profile, and finally the halo mass distribution function compared to the theoretical one (Press-Scheter). If you do not have Xquartz you can move the produced .png files to the /output folder to put it outside of the docker system.

6. Running with full physics

For the people interested we will also show how we can produce a simulation using full physics. In the parameter file, we will change the following parameters:

```
achOutName = L20_32_full
bDoGas = 1
bGasCooling = 1
bUV = 1
bStarForm = 1
bFeedBack = 1
```

After changing this we simply follow the previous setups outlined above. After the simulation is done, we do the same analysis/visualization as before, we can also output temperature to see the effect of feedback and also have a look at the distribution of star particles.

7. Compilation and options when running on HPC clusters

A general HPC build would look something like:

```
./build ChaNGa mpi-linux-x86_64 smp cuda --with-production -j8 --force; \
```

The option cuda determines if we run with GPU or not.